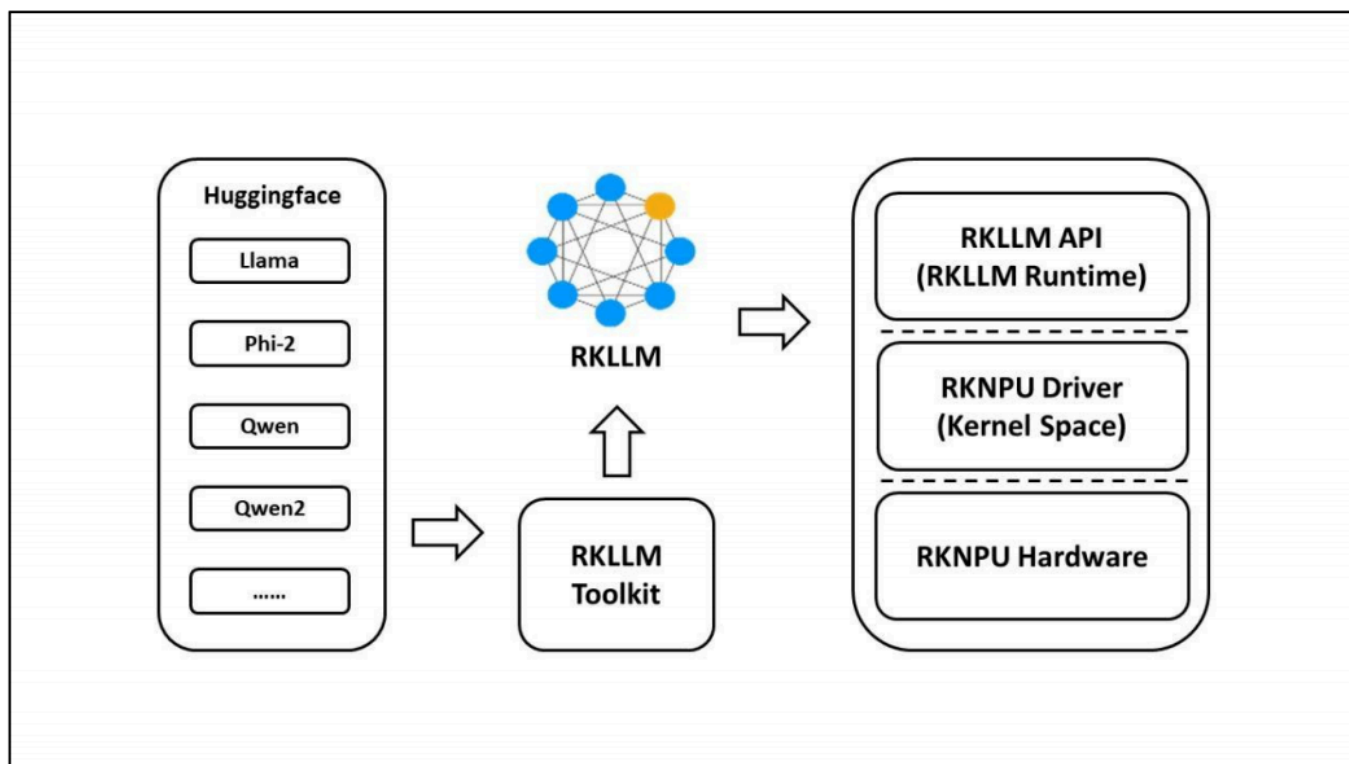


32.Deepseek/Qwen大模型部署

1. RKLLM基本介绍



官网文档地址：

https://github.com/airockchip/rknnllm/blob/main/doc/Rockchip_RKLLM_SDK_CN_1.2.3.pdf

1.1 RKLLM-Toolkit 功能介绍

RKLLM-Toolkit 是为用户提供在计算机上进行大语言模型的量化、转换的开发套件。通过该工具提供的 Python 接口可以便捷地完成以下功能：

1) 模型转换：支持将 **Hugging Face** 和 **GGUF** 格式的大语言模型（Large Language Model, LLM）转换为 RKLLM 模型，目前支持的模型包括 LLaMA, Qwen, Qwen2, Qwen3, Qwen2-VL, Qwen3-VL, Phi-2, Phi-3, ChatGLM3, Gemma, Gemma2, Gemma3, Gemma3n, InternLM2, TeleChat2, MiniCPM-S, MiniCPM, MiniCPM3, MiniCPM4, DeepSeekOCR, InternVL3，转换后的 RKLLM 模型能够在 Rockchip NPU 平台上加载使用。

2) 量化功能：支持将浮点模型量化为定点模型，目前支持的量化类型包括：

- w4a16;
- w4a16 分组量化(支持的分组数为 32, 64, 128);
- w8a8;

- w8a8 分组量化(支持的分组数为 128, 256, 512);

1.2 RKLLM Runtime 功能介绍

RKLLM Runtime 主要负责加载 RKLLM-Toolkit 转换得到的 RKLLM 模型，并在板端通过调用 NPU 驱动在 Rockchip NPU 上加速 RKLLM 模型的推理。在推理 RKLLM 模型时，用户可以自行定义 RKLLM 模型的推理参数设置，定义不同的文本生成方式，并通过预先定义的回调函数不断获得模型的推理结果。

1.3 RKLLM 开发流程介绍

RKLLM 的整体开发步骤主要分为 2 个部分：模型转换和板端部署运行。

1) 模型转换：

在这一阶段，用户提供 Hugging Face 格式的大语言模型将会被转换为 RKLLM 格式，以便在 Rockchip NPU 平台上进行高效的推理。这一步骤包括：

- a. 获取原始模型：**1、开源的 Hugging Face 格式的大语言模型；2、自行训练得到的大语言模型，要求模型保存的结构与 Hugging Face 平台上的模型结构一致；3、GGUF 模型，目前仅支持 q4_0 和 fp16 类型模型；
- b. 模型加载：**通过 `rkllm.load_huggingface()` 函数加载 huggingface 格式模型，通过 `rkllm.load_gguf()` 函数加载 GGUF 模型；
- c. 模型量化配置：**通过 `rkllm.build()` 函数构建 RKLLM 模型，在构建过程中可选择是否进行模型量化来提高模型部署在硬件上的性能，以及选择不同的优化等级和量化类型。
- d. 模型导出：**通过 `rkllm.export_rkllm()` 函数将 RKLLM 模型导出为一个 `.rkllm` 格式文件，用于后续的部署。

2) 板端部署运行：

这个阶段涵盖了模型的实际部署和运行。它通常包括以下步骤：

- a. 模型初始化：**加载 RKLLM 模型到 Rockchip NPU 平台，进行相应的模型参数设置来定义所需的文本生成方式，并提前定义用于接受实时推理结果的回调函数，进行推理前准备。
- b. 模型推理：**执行推理操作，将输入数据传递给模型并运行模型推理，用户可以通过预先定义的回调函数不断获取推理结果。
- c. 模型释放：**在完成推理流程后，释放模型资源，以便其他任务继续使用 NPU 的计算资源。

以上这两个步骤构成了完整的 RKLLM 开发流程，确保大语言模型能够成功转换、调试，并最终在 Rockchip NPU 上实现高效部署。

2. 搭建开发环境

2.1 安装 Miniconda

Conda 是一个开源的软件包管理系统和环境管理系统，它可以用于安装、管理和升级软件包和依赖项，我们这里使用conda 的目的只是构建一个虚拟环境，所以选择轻量化的miniconda。

miniconda 的官方链接如下所示：<https://www.anaconda.com/download/success>

Choose Your Download

Windows

Mac

Linux

Anaconda Distribution

Complete package with 8,000+ libraries, Jupyter, JupyterLab, and Spyder IDE. Everything you need for data science.

[⬇ 64-Bit \(x86\) Installer](#)

[⬇ 64-Bit \(AWS Graviton2 / ARM64\) Installer](#)

Miniconda

Minimal installer with just Python, Conda, and essential dependencies. Install only what you need.

[⬇ 64-Bit \(x86\) Installer](#)

[⬇ 64-Bit \(AWS Graviton2 / ARM64\) Installer](#)

Hi, how ca

下载Miniconda3-latest-Linux-x86_64.sh

代码块

```
1 wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
```

执行脚本

代码块

```
1 ./Miniconda3-latest-Linux-x86_64.sh
```

```
topeet@ubuntu:~/rknn/01_miniconda$ ls
Miniconda3-latest-Linux-aarch64.sh  Miniconda3-latest-Linux-x86_64.sh
topeet@ubuntu:~/rknn/01_miniconda$ chmod 777 Miniconda3-latest-Linux-x86_64.sh
topeet@ubuntu:~/rknn/01_miniconda$ ls
Miniconda3-latest-Linux-aarch64.sh  Miniconda3-latest-Linux-x86_64.sh
topeet@ubuntu:~/rknn/01_miniconda$ sudo bash Miniconda3-latest-Linux-x86_64.sh
[sudo] topeet 的密码:

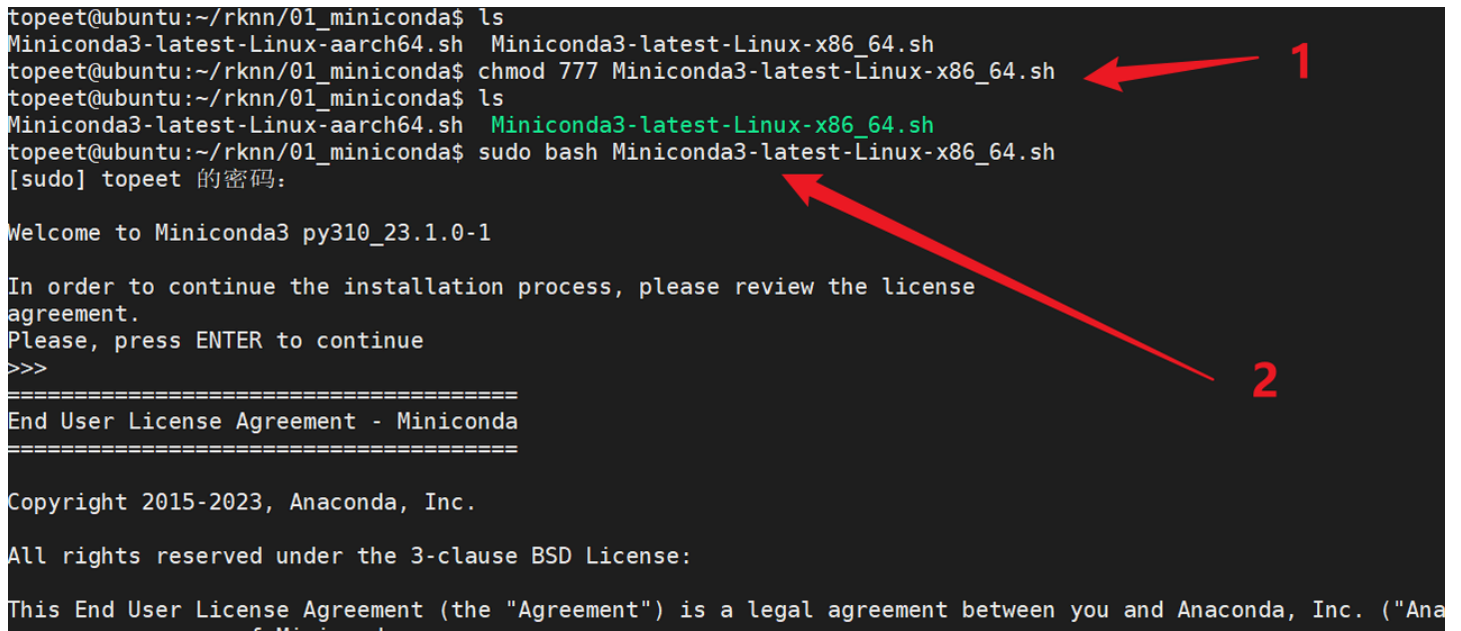
Welcome to Miniconda3 py310_23.1.0-1

In order to continue the installation process, please review the license
agreement.
Please, press ENTER to continue
>>>
=====
End User License Agreement - Miniconda
=====

Copyright 2015-2023, Anaconda, Inc.

All rights reserved under the 3-clause BSD License:

This End User License Agreement (the "Agreement") is a legal agreement between you and Anaconda, Inc. ("Ana
```



2.2 创建 RKLLM 虚拟环境

创建一个 Python3.10 版本（建议版本）名为 rkllm 的 Conda 环境

代码块

```
1 conda create -n rkllm python=3.10
```

进入 RKLLM-Toolkit Conda 环境

代码块

```
1 conda activate rkllm
```

安装 rkllm_toolkit

代码块

```
1 cd packages
2 pip install rkllm_toolkit-1.2.3-cp310-cp310-linux_x86_64.whl -i
  https://pypi.tuna.tsinghua.edu.cn/simple
```

看到 successful 即证明安装成功了。

```
Successfully built transformers-stream-generator
Installing collected packages: triton, sentencepiece, pytz, nvidia-cusparselt-cu12, mpmath, genson, flatbuffers, einops, easydict, addict, xxhash, urllib3, tzdata, typing-extensions, tqdm, toml, toml, tabulate, sympy, six, safetensors, rpsds-py, regex, pyyaml, pytokens, pyarrow, psutil, protobuf, propcache, platformdirs, pillow, pathspec, nvidia-nvtx-cu12, nvidia-nvjitlink-cu12, nvidia-nccl-cu12, nvidia-curand-cu12, nvidia-cufft-cu12, nvidia-cuda-runtime-cu12, nvidia-cuda-nvrtc-cu12, nvidia-cuda-cupti-cu12, nvidia-cublas-cu12, numpy, networkx, mypy-extensions, MarkupSafe, isort, inflect, idna, humanfriendly, hf-xet, fsspec, frozenlist, filelock, dnspython, dill, colorlog, click, charset-normalizer, certifi, attrs, async-timeout, argcomplete, annotated-types, aiohappyeyeballs, typing-inspection, scipy, rouge, requests, referencing, python-dateutil, pydantic-core, nvidia-cusparselt-cu12, nvidia-cudnn-cu12, multiprocessing, multidict, jsonlines, Jinja2, gekko, email-validator, coloredlogs, black, aiohttp, yarl, tiktoken, pydantic, pandas, nvidia-cusolver-cu12, jsonschema-specifications, huggingface-hub, torch, tokenizers, jsonschema, aiohttp, transformers, torchvision, datamodel-code-generator, accelerate, transformers-stream-generator, timm, peft, datasets, optimum, auto-gptq, rllm-toolkit
Successfully installed Jinja2-3.1.4 MarkupSafe-3.0.3 accelerate-1.5.2 addict-2.4.0 aiohappyeyeballs-2.6.1 aiohttp-3.13.3 aiohttp-3.13.3 annotated-types-0.7.0 argcomplete-3.6.3 async-timeout-5.0.1 attrs-25.4.0 auto-gptq-0.7.1 black-26.3.0 certifi-2026.2.25 charset-normalizer-3.4.5 click-8.3.1 coloredlogs-15.0.1 colorlog-6.8.2 datamodel-code-generator-0.27.0 datasets-4.1.1 dill-0.4.0 dnspython-2.8.0 easydict-1.13 einops-0.4.1 email-validator-2.3.0 filelock-3.25.2 flatbuffers-24.3.25 frozenlist-1.8.0 fsspec-2025.10.0 gekko-1.3.2 genson-1.3.0 hf-xet-1.4.0 huggingface-hub-0.36.2 humanfriendly-10.0 idna-3.11 inflect-5.6.2 isort-5.13.2 jsonlines-4.0.0 jsonschema-4.23.0 jsonschema-specifications-2025.9.1 mpmath-1.3.0 multidict-6.7.1 multiprocessing-0.70.16 mypy-extensions-1.1.0 networkx-3.4.2 numpy-1.26.4 nvidia-cublas-cu12-12.4.5.1 nvidia-cuda-cupti-cu12-12.4.127 nvidia-cuda-nvrtc-cu12-12.4.127 nvidia-cuda-runtime-cu12-12.4.127 nvidia-cudnn-cu12-9.1.0.70 nvidia-cufft-cu12-11.2.1.3 nvidia-curand-cu12-10.3.5.147 nvidia-cusolver-cu12-11.6.1.9 nvidia-cusparselt-cu12-12.4.1.170 nvidia-cusparselt-cu12-0.6.2 nvidia-nccl-cu12-2.21.5 nvidia-nvjitlink-cu12-12.4.127 nvidia-nvtx-cu12-12.4.127 optimum-1.23.3 pandas-2.3.3 pathspec-1.0.4 peft-0.18.1 pillow-10.4.0 platformdirs-4.9.4 propcache-0.4.1 protobuf-4.25.4 psutil-7.2.2 pyarrow-21.0.0 pydantic-2.12.5 pydantic-core-2.41.5 python-dateutil-2.9.0.post0 pytokens-0.4.1 pytz-2026.1 postl pyyaml-6.0.3 referencing-0.37.0 regex-2026.2.28 requests-2.32.5 rllm-toolkit-1.2.3 rouge-1.0.1 rpsds-py-0.30.0 safetensors-0.5.3 scipy-1.15.3 sentencepiece-0.2.0 six-1.17.0 sympy-1.13.1 tabulate-0.9.0 tiktoken-0.9.0 timm-1.0.19 tokenizers-0.21.4 toml-0.10.2 toml-2.4.0 torch-2.6.0 torchvision-0.21.0 tqdm-4.67.3 transformers-4.55.2 transformers-stream-generator-0.0.5 triton-3.2.0 typing-extensions-4.15.0 typing-inspection-0.4.2 tzdata-2025.3 urllib3-2.6.3 xxhash-3.6.0 yarl-1.23.0
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possibly rendering your system unusable. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv. Use the --root-user-action option if you know what you are doing and want to suppress this warning.
(rkllm) root@ubuntu:/home/topeet/rknn/07_rkllm/packages#
```

3. Deepseek/Qwen部署

3.1 下载模型

因为模型也比较大，去官网有很大概率下载失败，直接去魔搭社区下载。

<https://www.modelscope.cn/models/deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B/summary>

代码块

```
1 pip install modelscope
2
3 python -c "from modelscope import snapshot_download; model_dir =
snapshot_download('deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B',
cache_dir='~/rknn')"
```

3.2 模型转换

将所有文件放到对应的目录，如图所示。

```
topeet@ubuntu:~/rknn/07_rkllm$ ls
deepseek_1.5b_W8A8_RK3588.rkllm  DeepSeek-R1-Distill-Qwen-1.5B  packages  rknn-llm-main
topeet@ubuntu:~/rknn/07_rkllm$
```

拷贝官方demo到模型同一目录。

```
1  cp -r rknn-llm-main/rkllm-toolkit/examples/custom_demo/ ./
```

RKLLM 支持 INT8/W8A8 量化以提升推理速度。我们需要先生成校准数据 data_quant.json：

代码块

```
1  python3 generate_data_quant.py -m /home/topeet/rknn/07_rkllm/DeepSeek-R1-Distill-Qwen-1.5B
```

然后修改 export_rkllm.py 文件，将 modelpath 修改为自己模型存放的路径

代码块

```
1  modelpath = "DeepSeek-R1-Distill-Qwen-1___5B"
```

然后进行rkllm模型转换

代码块

```
1  python3 export_rkllm.py
```

注意：这里最好是在ubuntu系统或者有显卡的win也可以，虚拟机里面没办法调用显卡，基本上你转换不成功。

可以使用RK官方转换转换好的模型：

<https://meta.box.lenovo.com/v/link/view/ad7482f6712844b48902f07287ed3359> 提取码:rkllm

3.3 编译与构建

可以通过运行 **deploy/build-linux.sh** 脚本直接编译示例代码（请将脚本中的交叉编译器路径替换为实际路径）。

这将在 deploy 目录下生成一个 install/demo_Linux_aarch64 文件夹，其中包含可执行文件 llm_demo和lib文件夹。

代码块

```
1  cd deploy
2
3  # 修改交叉编译工具的路径,这里修改为你实际的就行
4  GCC_COMPILER_PATH=/opt/tool_chain/gcc-arm-10.3-2021.07-x86_64-aarch64-none-
    linux-gnu/bin/aarch64-none-linux-gnu
5
```

```
6 ./build-linux.sh
```

编译成功如下所示：

```
(rkllm) root@ubuntu:/home/topeet/rknn/07_rkllm/rknn-llm-main/examples/rkllm_api_demo/deploy# ls
build build-android.sh build-linux.sh CMakeLists.txt install
(rkllm) root@ubuntu:/home/topeet/rknn/07_rkllm/rknn-llm-main/examples/rkllm_api_demo/deploy# cd install/
(rkllm) root@ubuntu:/home/topeet/rknn/07_rkllm/rknn-llm-main/examples/rkllm_api_demo/deploy/install# ls
demo_Linux_aarch64
(rkllm) root@ubuntu:/home/topeet/rknn/07_rkllm/rknn-llm-main/examples/rkllm_api_demo/deploy/install# cd demo_Linux_aarch64/
(rkllm) root@ubuntu:/home/topeet/rknn/07_rkllm/rknn-llm-main/examples/rkllm_api_demo/deploy/install/demo_Linux_aarch64# ls
lib llm_demo
(rkllm) root@ubuntu:/home/topeet/rknn/07_rkllm/rknn-llm-main/examples/rkllm_api_demo/deploy/install/demo_Linux_aarch64#
```

3.4 开发板运行测试

将前面两个小节转换得到的大语言 RKNN 模型和编译得到的可执行文件拷贝到开发板上。

```
root@topeet:/rkllm#
root@topeet:/rkllm# ls
deepseek_1.5b_W8A8_RK3588.rkllm lib llm_demo
root@topeet:/rkllm#
```

由于加载大语言模型会打开很多的文件，默认情况每个进程可以同时打开的文件数是有限制的，所以需要在终端输入以下命令，允许当前用户在一个会话中打开最多 102400 个文件描述符。

代码块

```
1 ulimit -HSn 102400
```

然后使用以下命令，运行可执行程序，并加载 RKLLM 大语言模型，运行成功如下图所示：

代码块

```
1 export LD_LIBRARY_PATH=./lib
2 ./llm_demo deepseek_1.5b_W8A8_RK3588.rkllm 2048 4096
```

第一个参数 `2048`：最大上下文长度

- 含义：这是模型单次对话能“记住”的最大容量。它包含了你输入的问题 + 模型已经生成的回答的总和。
- 单位：Token（词元）。对于中文，1个 Token 大约对应 0.5 到 1 个汉字。
- 作用：
 - 如果你输入一篇很长的文章让模型总结，这个值必须足够大，否则文章会被截断，模型读不全。

- 在 RK3588 这种内存受限的设备上，这个值不能无限大。2048 是一个在 4GB 内存设备上比较安全的数值。如果设置成 8192 或更高，很容易导致内存溢出（OOM），程序会被系统直接杀掉。

第二个参数 4096：最大生成长度

- 含义：这是模型单次回复允许输出的最大 Token 数量。
- 作用：
 - 这是一个“止损”限制。如果模型开始滔滔不绝地写小说，或者陷入了死循环一直重复同一句话，达到这个数值后，程序会强制停止生成。
 - 注意：这并不代表模型一定会生成这么多字。如果模型觉得话说完了（遇到了结束符），它会提前停止。

user: 你是谁

robot: 我是一个AI助手，没有实体，但我可以和你聊天、提供信息、帮助解决问题。我是通义千问，由阿里云研发的超大规模语言模型。我可以回答各种问题，包括科技、文化、生活等领域的知识，也可以进行创作和对话。如果你有任何问题或需要帮助，请随时告诉我！

user: █